Ders 8 Genişletilmiş Notu: Bağlam Farkında Sinirsel Dil Modelleri

Ders 8 Genişletilmiş Notu: Bağlam Farkında Sinirsel Dil Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bölüm 5'e Giriş
- 2. Dağıtık Temsil (Distributed Representation)
- 3. Alt Özellikler Nasıl Öğrenilir ve Transfer Öğrenme
- 4. Kapsül Ağları (Capsule Networks)
- 5. Yinelemeli Sinir Ağları (RNN)
- 6. RNN Uygulama Türleri
- 7. RNN Denklemi ve Öğrenilen Ağırlıklar
- 8. RNN ile Bir Sonraki Harfi Tahmin
- 9. RNN Türevleri: LSTM ve GRU
- 10. RNN'lerle Dil Çevirisi: Kodlayıcı-Kod Çözücü
- 11. Dil Çevirisinde Bazı İyileştirmeler
- 12. Dikkat Mekanizması (Attention)
- 13. Transformer'lar ve Öz-Dikkat
- 14. Bağlam Tabanlı Dil Modelleri
- 15. Kendi Kendine Öğrenme Görevleri (Self-Supervised)
- 16. Alt Kelimeler ve Byte Pair Encoding
- 17. Alanın Genel Resmi
- Örnek Sorular
- Dersin Toparlaması

1. Bölüm 5'e Giriş

Önceki iki derste makine öğrenmesindeki yeni yaklaşımlar (Bölüm 4) tamamlanmıştı. Bu derste ders notlarının Bölüm 5'ine geçildi. Slaytın kapağındaki başlık "Bağlam Farkında Sinirsel Dil Modelleri"dir (ekrandan, 00:10:00). Hoca konuya doğrudan bağlam tabanlı modellerle girmek yerine, önce bu modellerin üzerine kurulduğu birkaç kavramı anlatacağını söyledi. Bunlar dağıtık temsil, RNN, kodlayıcı-kod çözücü mimarisi, dikkat mekanizması ve Transformer'dır. Hoca bunlara bağlam tabanlı modellerin "primitifleri, bileşenleri" dedi (01:31:08). Dersin son kısmında bağlam tabanlı dil modelleri, kendi kendine öğrenme görevleri ve alt kelime temsili ele alındı.

2. Dağıtık Temsil (Distributed Representation)

Hoca dağıtık temsili yapay sinir ağlarının en güçlü yanı olarak tanıttı. Slayttaki örnek şudur (ekrandan, 00:11:30): resimlerdeki insanları kadın/erkek, gözlüklü/gözlüksüz ve uzun/kısa diye belirlemek istiyoruz. Her kriterin iki değeri olduğu için $2 \cdot 2 \cdot 2 = 8$ farklı kombinasyon vardır. Bu problemi 8 sınıflı bir problem olarak ele alırsak her sınıftan örneğe ihtiyacımız olur: kısa, gözlüksüz kadın örneklerine; kısa, gözlüklü erkek örneklerine ve böyle devam eder.

Bunun yerine 3 alt özelliği öğrenen 3 ikili (binary) sınıflandırıcı tanımlayabiliriz: biri kadını erkekten, biri gözlüklüyü gözlüksüzden, biri uzunu kısadan ayırır. Bu durumda 8 sınıfı bulmaya çalışmıyoruz; 3 alt özelliği öğreniyoruz. Böylece eğitim kümesinde uzun, kadın ve gözlüksüz bir örnek hiç olmasa bile, elimizde uzun/kısa, kadın/erkek ve gözlüklü/gözlüksüz örnekleri olduğu sürece bu kombinasyonu tanıyabiliriz. 3 ikili sınıflandırıcıdan 8 farklı çıkış elde edilir.

Genel kural slaytta şöyle yazar: d adet alt özellikle 2^d farklı kombinasyon ifade edilebilir. Yani N adet sınıf, böyle örtüşmeli (non mutually exclusive) özellikler taşıyorsa $\log_2 N$ adet alt özellikle ifade edilebilir.

2.1 Sayısal örnek: parametre sayısı

Hoca bu fikrin parametre sayısına etkisini bir örnekle gösterdi (ekrandan, 00:15:10). Elimizde 1024 sınıflı bir problem ve $d \times d$ boyutlu resimler olsun. Ara katmanı olmayan, dümdüz bir yapıyı düşünelim.

- Klasik yaklaşım: 1024 prototip öğrenmemiz gerekir. Ağın yapısı $(d \cdot d) \cdot 1024$ olur; yani girişten çıkışa doğrudan $d \cdot d \cdot 1024$ parametre öğrenilir.
- Dağıtık yaklaşım: 10 alt özellik kullanırsak ağa bir katman eklenir ve yapı $(d \cdot d) \cdot 10 \cdot 1024$ olur. İlk kısım 10 alt özelliği öğrenir, son katman alt özelliklerin kombinasyonlarını sınıf etiketlerine eşlemeyi öğrenir. Öğrenilecek parametre sayısı $(d \cdot d) \cdot 10 + (10 \cdot 1024)$ olur.

$d = 128$ için hesap:

- Klasik: $128 \cdot 128 \cdot 1024 = 16384 \cdot 1024 = 16.777.216 = 2^{24}$.
- Dağıtık: $128 \cdot 128 \cdot 10 + 10 \cdot 1024 = 163.840 + 10.240 = 174.080$, yani yaklaşık 180 bin.

  Ders sonrası not: Slaytta ve derste klasik yaklaşım için "yaklaşık 4 milyon (2^{24})" denildi. 2^{24} doğrudur, ama $2^{24} = 16.777.216$, yani yaklaşık 16,8 milyondur, 4 milyon değildir (python ile hesaplandı). Dağıtık yaklaşım için verilen "yaklaşık 180 bin" doğrudur.

Sonuç olarak katman sayısını arttırdık (girişten bir saklı duruma, saklı durumdan çıkışa), ama öğrenmemiz gereken parametre sayısı önemli ölçüde azaldı. Hoca bunu 1024 sınıfı 10 özelliğin farklı kombinasyonları olarak görebilmek diye özetledi; dağıtık temsilin özü budur.

3. Alt Özellikler Nasıl Öğrenilir ve Transfer Öğrenme

Sekiz sınıfı üç özellik, 1024 sınıfı on özellik belirliyorsa, bu alt özellikleri nasıl belirleyeceğiz? Slayt bunu sorup "Öğrenelim" diye cevaplar (ekrandan, 00:20:00). Alt özellikleri elle tanımlanmış kurallarla belirlemek kolay değildir; bunun yerine uçtan uca öğrenme (end-to-end learning) ile öğrenilirler. Bir MLP'nin her gizli katmanındaki her nöron bir alt özellik öğrenir. Bir CNN'deki her filtre bir alt özellik öğrenir.

Slayttaki varsayım şudur: tüm resimler ortak küçük parçalardan oluşur. En alt seviyede basit özellikler, sonrasında bunların kombinasyonları öğrenilir. Slayttaki görselde yüzler, arabalar, filler ve sandalyeler için eğitilmiş dört ağın katmanları vardır. En alt katmanlardaki özellikler dördünde de neredeyse aynıdır: yatay çizgi, dikey çizgi, tamamen dolu ya da tamamen boş bölgeler gibi. Hoca bunu, bir resme mikroskobik ölçekte bakıldığında hep bu tür ilkel parçaların görüleceğiyle açıkladı. Üst katmanlarda bu parçalar birleşerek probleme özgü bileşenlere dönüşür.

Kişileri birbirinden ayırt etmek (bu Ahmet'tir, bu Mehmet'tir) için tasarlanmış bir ağ ile arabaları ayırt eden bir ağ, alt seviyelerde birbirinden çok farklı özellikler öğrenmez; farklılaşma üst seviyelerde olur. Bu, dört problem ayrı ağlarla da eğitilse, dört sınıflı tek bir problem olarak tek ağla da eğitilse gözlenir.

Bu gözlem öğrenmenin transferinin (transfer learning) neden mümkün olduğunu açıklar. Farklı problemlerde alt seviyelerde hep benzer şeyler öğreniliyorsa, bu alt seviye temsiller çok geniş veri kümeleriyle bir kez öğrenilebilir. Yeni bir görev geldiğinde yalnız o göreve özgü üst seviyeler farklılaştırılır.

4. Kapsül Ağları (Capsule Networks)

Hoca kapsül ağlarından yalnız bir kavram olarak bahsetti (ekrandan, 00:25:30). Bu yaklaşımı Hinton önermiştir. Slayttaki açıklamaya göre CNN'de küçük parçaların resmin herhangi bir yerinde var olması yeterlidir. Pooling katmanı parçanın nerede olduğunu önemsizleştirir; sadece olup olmadığını söyler.

Oysa bu parçaların birbirlerine göre konumları da önemlidir. Bir yüz resminde ağızla gözün yerini, burunla gözün yerini değiştirirseniz bir CNN hâlâ "yüz" diyebilir, ama bir insan burada bir anormallik olduğunu fark eder. CNN'lere alternatif olan kapsül ağları hem alt özellikleri hem de bu alt özelliklerin birbirine göre konumlarını birlikte öğrenir. Ayrıntısına girilmedi.

5. Yinelemeli Sinir Ağları (RNN)

Hoca RNN'leri bu ders için çok kritik mimarilerden biri olarak tanıttı. Temel özellikleri, girişin bir sekans olduğu uygulamalar için tasarlanmış olmalarıdır (ekrandan, 00:30:00). Burada önemli olan, girişin sekans olmasının yanında sekanslığının da dikkate alınmasıdır. Giriş bir sekans olabilir ama onun sırası göz ardı edilebilir; RNN'lerde ise sıra göz önüne alınarak işlenir.

Sıranın önemli olduğu girişlere örnekler:

- Bir cümlenin kelimeleri: kelimelerin yeri önemlidir. Türkçe gibi dillerde bazı sıra değişiklikleri kabul edilir, ama bütün varyasyonlar kabul edilmez.
- Bir filmin kareleri: bir saniyenin içindeki kareleri rastgele karıştırırsanız (çok hızlı bir aksiyon filmi değilse) belki pek bir şey kaybetmezsiniz, ama üç saniyelik kareleri karıştırırsanız çok garip sonuçlar çıkar.
- Bir konuşmanın fonemleri, yani ses sinyali.

5.1 Gösterim

Slayttaki tanımlar şöyledir: x_t, t. andaki giriştir; h_t, t. andaki çıkıştır. h_t, x_t'ye ve h_{t-1}'e bağlıdır; aslında $x_0 \dots x_t$'nin hepsine bağlıdır. RNN bloğu ürettiği h_t'yi hem dışarı verir hem de bir sonraki girişte etkili olsun diye kendisine geri gönderir. Açılmış (unrolled) çizimde x_0 girer, h_0 çıkar; h_0 ve x_1 girer, h_1 çıkar; h_1 ve x_2 girer, h_2 çıkar ve böyle devam eder.

5.2 Neden bütün geçmişe bağlı?

Hoca bunu adım adım açtı (ekrandan, 00:40:00):

$$h_t = f(x_t, h_{t-1})$$

$$h_3 = f(x_3, h_2), \quad h_2 = f(x_2, h_1), \quad h_1 = f(x_1, h_0)$$

h_2 yerine tanımını koyarsak:

$$h_3 = f(x_3, f(x_2, h_1))$$

h_1 yerine de tanımını koyarsak:

$$h_3 = f(x_3, f(x_2, f(x_1, h_0)))$$

Böylece h_3'ün x_3'e, x_2'ye, x_1'e ve h_0'a bağlı olduğu açıkça görülür. Yani herhangi bir andaki çıkış o ana kadarki bütün girişlere bağlıdır.

5.3 Sabit uzunluklu ağlarla karşılaştırma

Hoca aynı işin RNN olmadan, standart bir yapay sinir ağıyla da yapılabileceğini gösterdi. Klasik bir yapay sinir ağının girişi uzunluk bakımından sabit olmak zorundadır. Slayttaki örnek (ekrandan, 00:37:40):

```
Input  = 5 kelime
Output = next word

Ali bugün okula giderken yolda arkadaşı Hilmiyi gördü.

Ali bugün okula giderken yolda        -> arkadaşı
bugün okula giderken yolda arkadaşı   -> Hilmiyi
okula giderken yolda arkadaşı Hilmiyi -> gördü
```

Bir cümleden, beş kelimelik sabit bir pencere kaydırılarak sonraki kelimeyi tahmin etmek için yeni eğitim örnekleri üretilir. Slayttaki ifadeyle, bu gösterim klasik ağların da sabit büyüklükte geçmişi işe katmalarının sağlanabileceğini gösterir: giriş x yerine birleştirilmiş $(x_t, x_{t-1}, \dots, x_{t-k})$ olarak verilir. Böylece y_t sadece x_t'ye değil, geçmişine de bağlı olarak modellenebilir. Son zamanlarda RNN'lerin işini yapan Transformer'lar da böyle sabit bir uzunlukla çalışır.

Fark şudur: sabit yapıda bir uzunluk söylemek zorundasınız (3, 5, 7, 10, 100, 1000), çünkü mimari sabittir. Beş kelimelik pencerede h_5 önceki beş kelimeye bağlı olabilir, ama h_6 daha geriye gidemez; yukarıdaki örnekte "Ali" son tahmini etkileyemez. RNN'de ise girişler teker teker verilir ve t. andaki çıkış başlangıçtan itibaren girilen her şeyin fonksiyonudur. Slaytın ifadesiyle RNN'in uzunluk sınırı sonsuzken, sabit yapılarda bir sınır vardır. Hoca pratikte pencere yeterince geniş tutulursa sabit yapının oldukça iyi çalıştığını, ama teorik olarak RNN'de çok eski bir girişin bile şimdiki çıkışı etkileyebileceğini vurguladı.

6. RNN Uygulama Türleri

Slayttaki dört tür şunlardır (ekrandan, 00:42:00):

Tür	Örnek
one to one	Klasik sinir ağı: bir şey girer, bir şey çıkar
one to many	Resim girer, onu anlatan cümle çıkar
many to one	Tweet girer, duygusu (sentiment) çıkar
many to many	Cümle girer, başka bir dildeki karşılığı çıkar

Many to one türünde kelimeler teker teker gelir ve sonunda tek bir sınıf ortaya çıkar. Many to many (çeviri) türünde birinci cümle bittikten sonra başka dildeki karşılığı üretilir; giriş de çıkış da sekanstır. One to many için hoca şu benzetmeyi yaptı: "kedi" dersiniz, model size adım adım, piksel piksel bir kedi üretir.

7. RNN Denklemi ve Öğrenilen Ağırlıklar

RNN'lerin temel eşitliği slaytta şöyledir (ekrandan, 00:45:00):

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

h_{t-1} bir W ile, x_t başka bir W ile çarpılır, ikisi toplanır ve bir aktivasyon fonksiyonundan geçirilir; sonuç h_t'dir. h, o ana kadar gelen bütün girişleri birden temsil eden bir vektördür; 3, 300 ya da 500 boyutlu olabilir. Slayttaki ifade: 3 saklı nöronlu bir RNN 3 boyutlu bir durum (state) tutar; yeni girişler geldikçe bu durumlar güncellenir. Durumlar, o ana kadar gelen girişlerin temsilidir. Öğrenilen ağırlıklar giriş ile saklı katman (durum) arası (W_{xh}) ve saklı katmanların t ile $t-1$ arası (W_{hh}) ağırlıklardır.

7.1 Sonlu durum makinesi benzetmesi

Hoca öğrencilere sonlu durum makinelerini (finite state machines) hatırlattı. Orada sonlu sayıda durum ve bir girişin bir durumu başka bir duruma geçirmesi vardı. RNN'de de olay budur, ama durum sayısı sabit değildir ve durum "birinci durum, beşinci durum" gibi ayrık değil, sürekli bir uzayda bir vektördür.

Duruma iki şey etki eder:

- W_{hh}: mevcut durumdan, girişe bağlı olmadan hangi duruma geçileceğini modeller; saklı katmandan saklı katmana geçiş matrisidir.
- W_{xh}: girişe bağlı kısımdır; giriş buyken hangi duruma geçileceğini modeller.

İki matris de yapının içinde öğrenilir. Her biri yeni bir h önerir; biri yalnız geçmişe, diğeri yalnız mevcut girişe bakarak. Bu iki etki toplanarak birleştirilir. Böylece h_t'de x_0'dan x_t'ye kadar bütün girişlerin birlikte bir temsili oluşur.

8. RNN ile Bir Sonraki Harfi Tahmin

Slayttaki örnek (ekrandan, 00:50:00): giriş ve çıkış 4 boyutludur (one-hot encoding, sözlük 4 karakter), 3 saklı nöron vardır ve RNN'in üzerine bir katman daha eklenmiştir; W_{hy} de öğrenilir. Şekildeki değerler şunlardır:

```
input chars :   "h"         "e"         "l"         "l"
input layer :  [1,0,0,0]  [0,1,0,0]  [0,0,1,0]  [0,0,1,0]
                    | W_xh
hidden layer:  [0.3,-0.1,0.9] -> [1.0,0.3,0.1] -> [0.1,-0.5,-0.3] -> [-0.3,0.9,0.7]   (W_hh)
                    | W_hy
output layer:  [1.0,2.2,-3.0,4.1] [0.5,0.3,-1.0,1.2] [0.1,0.5,1.9,-1.1] [0.2,-1.5,-0.1,2.2]
target chars:   "e"         "l"         "l"         "o"
```

Burada sözlüğün sırası h, e, l, o'dur.

Böylece bir RNN içinde öğrenilen üç şey vardır: girişi saklı duruma çeviren W_{xh}, önceki saklı durumu şimdiki saklı duruma çeviren W_{hh} ve saklı durumu çıkışa çeviren W_{hy}. Saklı durum 300 ya da 1000 boyutlu bir vektördür; onu bir kelimeye dönüştürmek için çıkışta softmax gibi bir yapı kullanılır.

Slayttaki soru şuydu: bir sonraki harf yerine bir sonraki kelimeyi tahmin etmek için one-hot kullansak ne olur, alternatifi nedir? Hoca, gerçekte sözlüğün 4 karakter olmayacağını, karakter düzeyinde bile yüze yakın karakter gerekeceğini, kelimelere geçildiğinde ise boyutun çok artacağını söyledi. Karakter ile kelimenin iyi yanlarını birleştiren yapı alt kelimelerdir (Bölüm 16).

9. RNN Türevleri: LSTM ve GRU

Aradan sonra hoca RNN'in farklı versiyonlarına geçti. Slayttaki listede LSTM, BiLSTM, GRU, NTM gibi türevler vardır. Hoca bunların hepsinin RNN olduğunu vurguladı: "GRU başka bir şey, LSTM başka bir şey, RNN başka bir şey" değildir. RNN büyük ailedir; LSTM ve GRU bu ailenin, RNN'lerin yapamadığı bazı şeyleri yapabilen alt türleridir.

> **Hocanın vurgusu:** Bir LSTM, bir GRU ya da çift yönlü bir LSTM gördüğünüzde bunun bir RNN olduğunu bilin.

9.1 Neyi çözüyorlar?

Temel RNN'de h_3, x_3'e, x_2'ye ve x_1'e, yani önceki bütün kelimelere bağlıdır. Ama her şeyin her şeye bağlı olması gerekmez. Örneğin bir paragrafın tamamını verdiğinizde paragrafın başındaki şeyler sonrakileri pek etkilemeyebilir; bir yerden sonra geçmişi aklında tutmaya gerek yoktur, bir yerde sıfırlamak gerekebilir. Bazen de sıfırlamadan devam etmek gerekir. LSTM'ler ve onun daha az parametre içeren versiyonu GRU, bir çizgi üzerinde nereye kadarki kısmın hatırlanıp nereden sonrasının unutulacağını, yani bazı x_t'lerin h'ye etki etmemesini sağlayan

yapılardır. Hoca bunları RNN'den çok büyük farkları olmayan, "bir tık daha uyarlanabilir" yapılar olarak niteledi: etkileme mekanizmasını dinamik olarak öğrenirler.

9.2 GRU denklemleri

Slayttaki GRU denklemleri şunlardır (ekrandan, 01:17:40):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Slayttaki notlar: GRU, LSTM'e göre daha basittir; 3 W matrisi (W_z, W_r, W) öğrenilir. Hoca bunların denklemlerinin ayrıntısına girmedi; bütün hikâyenin yine W'lar ve sigmoid fonksiyonları olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Büyük olasılıkla yaygın adlandırmayla, GRU'da z_t güncelleme kapısı (update gate), r_t sıfırlama kapısı (reset gate) olarak adlandırılır. z_t sıfıra yakınsa önceki durum olduğu gibi korunur, bire yakınsa yeni aday durum $\tilde{h}_t$ alınır. r_t sıfıra yakınsa aday durum hesaplanırken geçmiş büyük ölçüde unutulur.

10. RNN'lerle Dil Çevirisi: Kodlayıcı-Kod Çözücü

RNN'lerin en temel kullanımlarından biri dil çevirisi, daha genel olarak diziden diziye (sequence to sequence) dönüşümlerdir. Çevirinin önemli bir özelliği vardır: birinci cümlenin bütün kelimelerini görmeden ikinci cümlenin ilk kelimesini söylemek pek anlamlı değildir. Diller gramer olarak aynı değildir; cümlenin başı verildiğinde karşılığının başını üretmek her zaman mümkün olmaz. Bu yüzden önce kaynak cümle biter, sonra hedef cümle üretilir.

Bunun için görülen en basit mimari kodlayıcı-kod çözücüdür (encoder-decoder). Kodlayıcı bir dildeki cümleyi alıp bir vektöre dönüştürür; kod çözücü o vektörü alıp başka bir dildeki cümleye dönüştürür. Slayttaki denklemler şunlardır (ekrandan, 01:05:30):

Kodlayıcı:

$$h_t = \varphi(h_{t-1}, x_t) = f(W^{hh} h_{t-1} + W^{hx} x_t)$$

Kod çözücü:

$$h_t = \varphi(h_{t-1}) = f(W^{hh} h_{t-1})$$

$$y_t = \text{softmax}(W^S h_t)$$

Slayttaki şekilde Almanca "Echt dicke Kiste" kodlanır ve İngilizce "Awesome sauce" üretilir.

Kodlayıcıya kelimeler teker teker verilir. Örneğin 10 kelime verildikten sonra elde 300 boyutlu bir h kalır; bu vektör o ana kadar verilen bütün cümlenin A dilindeki temsilidir. Kod çözücüde yeni bir giriş (x) yoktur; bir sonraki adım yalnız bir önceki adıma bağlıdır. Elde edilen h_t'nin bir y_t'ye, yani bir kelimeye dönüşmesi gerekir; bunu W^S yapar.

Hoca W^S'yi word2vec'in ikinci yarısına benzetti. word2vec'te one-hot bir giriş uzayı, bir saklı katman ve one-hot bir çıkış uzayı vardı. Saklı katmandan çıkışa giden kısım, elinizdeki 300

boyutlu vektörün gerçekte hangi kelime olduğunu bulmaya çalışır; sözlük boyutu 32 bin ise 300'den 32 bine dönüştüren bir yapıdır ve softmax ile gerçekleştirilir.

Bu mimaride öğrenilenler kodlayıcıdaki iki matris (W^{hx} ve W^{hh}), kod çözücüdeki W^{hh} ve çıkıştaki W^S'dir. Kodlayıcıdaki ve kod çözücüdeki matrislerin birbirinin aynısı olması gerekmez.

11. Dil Çevirisinde Bazı İyileştirmeler

Hoca bu en basit yapının çok zor bir görevi üstlendiğini söyledi: girişteki bütün cümle tek bir vektöre dönüştürülüyor ve sonra o tek vektör, sadece W^{hh}'larla, sıradaki kelimeyi üretecek vektörlere dönüştürülüyor. Slayttaki ifadeyle tüm cümle tek bir duruma (C) indirgenmiştir (ekrandan, 01:10:00). Bunu kolaylaştırmak için çeşitli yaklaşımlar uygulanmıştır.

11.1 Cümleyi tersten vermek

Slayttaki ifade: kodlayıcıya cümleyi tersten verelim; kodlanacak ve çözülecek bilgiler arasındaki mesafeyi azaltalım. Örneğin w_1, w_2, w_3, w_4 cümlesinden a_1, a_2, a_3, a_4 cümlesi üretilmek istensin. Normal sırada C elde edildiğinde son görülen kelime w_4'tür, ama üretilecek ilk kelime a_1'dir ve w_1 ile ilgilidir. Giriş w_4, w_3, w_2, w_1 diye ters verilirse C en son w_1'den etkilenmiş olur ve cümlenin başını üretmek biraz daha kolaylaşır. Hoca ortalama uzaklık açısından çok fark olmadığını, ama cümlenin başının daha iyi üretilmesi sayesinde bir miktar iyileşme bildirildiğini, bunun çok büyük bir şey olmadığını söyledi.

11.2 Kod çözücüye C ve y_{t-1}'i vermek

Slayttaki kod çözücü denklemi şudur:

$$h_t = \varphi(h_{t-1}, C, y_{t-1})$$

Her biri için ayrı bir W vardır. Çıkış da şöyle hesaplanır:

$$y_t = \text{softmax}\left(W^S \begin{bmatrix} h_t \\ C \end{bmatrix}\right)$$

Yani kod çözücüye her adımda yalnız h_{t-1} değil, bütün cümleyi temsil eden C (kodlayıcının bittiği andaki vektör, t ile değişmez) ve bir önceki adımda üretilen kelime y_{t-1} de verilir. Hoca bunu "şunu hiçbir zaman unutma; bir de en son şunu ürettin" demeye benzetti.

Teoride bu eklemelerin bir anlamı yoktur: C zaten en baştaki durumdur ve y_{t-1} zaten h'den üretilmiştir. Kendisinden üretilen bir şeyi ona yeniden vermek gereksiz görünebilir; temel yapı evrensel olarak yeterlidir. Ama bu yaklaşımların pratikte işi kolaylaştırdığı görülmüştür.

Hoca makine çevirisinin pratikte, ticari olarak çok önemli bir problem olduğunu ve doğal dil işlemenin lokomotif problemlerinden biri olduğunu söyledi; ilerleyen konularda bunun nedeni görülecektir.

12. Dikkat Mekanizması (Attention)

Hoca dikkat (attention) denince akla Transformer'ların geldiğini, ama dikkatin Transformer'lardan önce de var olduğunu belirtti. LSTM ve GRU'da girişin bir yere kadarki kısmının etkilemesi, sonrasının etkilememesi gibi doğrusal bir yapı vardı. Dikkatte ise girişin hangi parçasının ne kadar etki etmesi gerektiği öğrenilir.

Slayttaki tanım (ekrandan, 01:20:00): kodlayıcının tüm durumları etkindir, ama ne ölçüde etkin (α) olacakları öğrenilir.

$$\alpha^{i,j} : \ i.\ \text{çıkışa}\ j.\ \text{girişin ne kadar etkisi olacağı}$$

$$\sum_t \alpha^{1,t} = 1$$

$$h^t = (h^{\rightarrow t}, h^{\leftarrow t})$$

$$C_1 = \sum_t \alpha^{1,t} h^t$$

Şekilde kodlayıcı çift yönlüdür: her giriş için ileri ve geri yönlü durumlar birleştirilerek h^t oluşturulur. Her çıkış için ayrı bir bağlam vektörü (C_1, C_2, ...) bu durumların ağırlıklı toplamı olarak hesaplanır ve kod çözücünün s_t durumuna verilir.

Slayttaki animasyonda Çinceden İngilizceye çeviri yapan, dikkat mekanizmalı bir RNN gösterildi. Girişteki kelimelerle çıkıştaki kelimeler arasındaki çizgilerin kalınlığı, girişteki o kelimenin çıkıştaki o kelimeye ne kadar etki ettiğini gösterir. Örneğin çıkışın bir kelimesini en çok e_5 ve e_6, ilk kelimesini e_0 ve e_1, ikinci kelimesini e_2 ve e_3 etkilemiştir.

α değerleri, önceki matrislere ek olarak mimarinin öğrendiği şeylerdir; bu yüzden parametre sayısı artar. Ama dikkat eklemenin performansa katkısı, Transformer'lardan önce bile çok büyük olmuştur. Bütün hikâye, hangi x'lerin y'nin hangi parçası üzerinde çok etkili olacağını öğrenmektir.

13. Transformer'lar ve Öz-Dikkat

Dikkatin büyük başarısından sonra "Attention is all you need" makalesi çıktı. Hoca bunun derin öğrenmenin mimarisini çok uzun süre değiştiren bir makale olduğunu ve tamamen bir makine çevirisi makalesi olduğunu vurguladı; Transformer mimarisi bu makalede önerildi ve teorik olarak bugünkü hali o zamankinden çok farklı değildir. Slayttaki maddeler şunlardır (ekrandan, 01:23:00):

- Attention is all you need.
- RNN yok. Dolayısıyla paralelleştirilebilir.
- Giriş ve çıkış dizisi arasındaki bağlılıkları klasik attention buluyor.
- Transformer'lar girişin ve çıkışın kendi içindeki bağımlılıklarını da işe katıyor.
- Multihead attention.

13.1 RNN olmaması

"RNN yok" ifadesi, Bölüm 5.3'teki sabit uzunluklu yapıyla aynı fikirdir: sekans teker teker değil, tek seferde ağa verilir. Teker teker verilseydi sonsuz uzunlukta yapılar elde edilebilirdi; birlikte verildiğinde en eskiler kalmaz, ama yeterince uzun bir pencerede pratikte büyük bir sıkıntı olmaz. Kelimelerin sırasının önemli olduğunu anlatmak için konum kodlaması (position encoding) kullanılır; bu, kelimenin cümlenin kaçıncı sırasında bulunduğunu ifade eden ek bir yapıdır.

13.2 Öz-dikkat (self-attention)

Hoca asıl kritik noktanın bu olduğunu söyledi. Transformer'lara kadar dikkat, girişin çıkışa etkisiydi. Transformer'lar öz-dikkati tanıttı: bir cümlenin içindeki kelimelerin yine o cümlenin içindeki kelimelere etkisi nedir?

Bunun çözdüğü problem, anlamı içinde kullanıldığı cümleyle ortaya çıkan kelimelerdir. Hoca bunu belki bütün kelimeler için söyleyebileceğimizi, ama bazı kelimeler için çok daha belirgin olduğunu söyledi. Slayttaki örnek (ekrandan, 01:30:00):

- "Garajımda 300'le gidebilen bir jaguar var."
- "Ormanda beni jaguar kovaladı."

İki cümledeki "jaguar" farklı şeylerdir. Birincisinde hangi jaguar olduğunu garaj ve 300'le gidebilmek; ikincisinde orman ve kovalamak belirler. Yani bir kelimenin nasıl temsil edileceği, içinde geçtiği diğer kelimelere de bağlıdır.

> **Hocanın vurgusu:** Çığır açan şey giriş-çıkış dikkatinden dizinin kendi içindeki dikkate geçiştir. Multihead attention ve key-value yapıları bunu gerçeklemenin pratik yollarıdır; ileride değişebilirler, ama özü öz-dikkattir.

Hoca key-value işlerinin nasıl yürüdüğünün ayrıntılarını kendi ders sayfasındaki Transformer kursunda bulunabileceğini söyledi. Key-value yapısının uzun süredir değişmemesinin, bunun güçlü bir yapı olduğunu gösterdiğini ekledi. Multihead attention aynı şeyin birden fazla temsilini oluşturur; 8 başlık (head) varsa, örneğin "jaguar"ın 8 temsili olur. Hoca bunun bir topluluk (ensemble) oluşturmaya benzediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Büyük olasılıkla bilinen standart tanımıyla, öz-dikkatte her kelime vektöründen öğrenilen üç matrisle sorgu (query), anahtar (key) ve değer (value) vektörleri üretilir. Bir kelimenin yeni temsili, onun sorgusuyla diğer kelimelerin anahtarlarının benzerliğinden (softmax ile normalize edilmiş) hesaplanan ağırlıklarla değer vektörlerinin ağırlıklı toplamıdır. Bu, Bölüm 12'deki $C = \sum \alpha h$ fikrinin dizinin kendi içine uygulanmasıdır.

14. Bağlam Tabanlı Dil Modelleri

Bağlam tabanlı dil modelleri, kelimeleri vektörlerle temsil eden ve bunları önceden eğitip (pretrain) saklayan word2vec gibi modellerin ötesine geçen yapılardır. Fark şudur: bağlam tabanlı bir modelde bir kelimenin vektörünün ne olacağı, kelime bir cümlenin içinde kullanılmadan belli değildir. word2vec'te ise "jaguar"ın tek bir vektörü vardır ve bu vektör hem araba olan jaguara hem ormanda gezen jaguara yakındır; bu yüzden çok doğru bir temsil değildir. word2vec'te her kelimenin vektörü sabittir ve bağlamdan bağımsızdır. Bağlam, kelimenin içinde geçtiği cümledir (kontekst). Resim için bağlam düşünülürse, bir videodaki önceki ve sonraki birkaç saniye gibi düşünülebilir.

14.1 Bağlamın etkisi: çizim örneği

Hoca bağlamın önemini göstermek için ekrana bir çizim yaptı (ekrandan, 01:34:20). Yatay sırada "11 12" ve ortada aynı şekil; dikey sırada "A" ve "C" ve yine ortada aynı şekil vardır. Ortadaki şekil yatay bağlamda (11, 12, …) 13, dikey bağlamda (A, …, C) B olarak görülür. İnsan bunu otomatik yapar, gidip gelme bile yaşamaz; bağlam baskındır.

word2vec'te bu şeklin (kelimenin) tek bir temsili vardır, bu da tek bir anlama gelmesi demektir. Bağlam tabanlı modelde temsilin bulunduğu bağlama göre değişmesi istenir. Bunun bedeli, kelime vektörlerinin bir yerde saklanamamasıdır. "Jaguar'ın vektörü budur, sakla" denemez. Elde bir model olur; cümle modele verilir ve "jaguar"ın o cümledeki vektörü hesaplanır. Bu vektör bağlama göre değişir.

14.2 Tarihçe ve modeller

Slayttaki liste şöyledir (ekrandan, 01:40:00):

- RNN tabanlı yaklaşımlar
  - TagLM: önceden eğitilmiş (sonraki kelimeyi tahmin için eğitilmiş) modelin başka bir görevde ilk kullanımı.
  - ELMo: çok sayıda görevde performans artışı.
- Transformer tabanlı yaklaşımlar
  - GPT: sonraki kelimeyi tahmin.
  - BERT: maskelenmiş kelimeyi tahmin; sonraki cümle mi ikili sınıflandırma (pozitif: ardışık iki cümle, negatif: rastgele iki cümle).
  - RoBERTa: maskelenmiş kelimeyi tahmin.
  - ALBERT: maskelenmiş kelimeyi tahmin; sonraki cümle mi ikili sınıflandırma (pozitif: ardışık iki cümle, negatif: pozitifteki 2 cümlenin yeri değiştirilmiş).
  - ELECTRA: değiştirilmiş kelimeleri tahmin.
  - BART: çeşitli görevler.

Hoca bağlam tabanlı modellerin Transformer'la başlamadığını, ilk yaklaşımların RNN tabanlı olduğunu anlattı. TagLM, sonraki kelimeyi tahmin etmek için önceden eğitilmiş bir modelin

başka bir görevde ince ayar (fine-tune) ile kullanıldığı ilk yapıdır; hoca bunu öğrenme transferinin NLP'deki ilk kullanımı olarak niteledi, ama tek bir görevde. ELMo da önceden eğitilmiş bir modeldir; güzel tarafı, bu önceden eğitilmiş modelin birçok farklı görevde performans artışı sağlayabileceğini göstermesidir. Yani yalnız elimizdeki eğitim kümesiyle bir model eğitmek yerine, önce çok büyük bir veriyle eğitilmiş bir model alınır, sonra kendi görevimiz ve veri kümemiz için ince ayar yapılır.

Ardından Transformer tabanlı modeller geldi; ilk ana noktası belki BERT'tir. Hoca GPT ile BERT'in çıkışları arasında zaman bakımından çok büyük fark olmadığını, ikisinin de temelde Transformer olduğunu ve ikisinin de kendi kendine öğrenmeyle eğitildiğini söyledi. GPT sonraki token'ı, BERT maskelenmiş token'ı tahmin eder. BERT'te ayrıca cümleler arası ilişkiyi öğrenmeye yönelik "sonraki cümle mi?" görevi vardır; GPT'de ve RoBERTa'da bu ek görev yoktur. ALBERT'te bu görev değiştirilmiştir, ama hoca bunun çok büyük farklar yaratmadığını söyledi. ELECTRA'da görev maskelenmiş kelimeyi değil, değiştirilmiş kelimeyi tahmin etmektir. Hoca bu modellerin güncel hallerini dönem sonuna doğru daha ayrıntılı göreceklerini söyledi.

15. Kendi Kendine Öğrenme Görevleri (Self-Supervised)

Bu görevlerin en önemli özelliği kendi kendine öğrenme olmalarıdır: insan etiketlemesine, insan emeğine ihtiyaç duyulmaz. Örneğin bir tweet'in pozitif mi negatif mi olduğunu bilmek için bir insanın etiketlemesi gerekir. Ama bir token'ı tahmin etmek için insanların yazdığı metinler yeterlidir. Metinler elimizdeyse "şu sekanstan sonra şu gelir" biçimindeki eğitim kümesi otomatik olarak üretilebilir.

Hoca bunun önemini şöyle açıkladı: birçok derin öğrenme mimarisi, modelleri çok büyük olduğu için ancak çok büyük miktarda veriyle iyi çalışabiliyor; bunun nedenleri henüz tam bilinmiyor. Bu büyüklükte veriyi insanlara etiketletmek mümkün değildir. Veri kümesini üretmek için insan etiketine ihtiyaç duyulmayınca ölçeklenebilirlik problemi aşılmış olur: bütün web sayfaları, bütün kitaplar kolayca eğitim kümesine dönüştürülebilir. Hoca kendi kendine öğrenmeyi son zamanların en kritik yaklaşımlarından biri saydı.

"Attention is all you need" makalesinde kendi kendine öğrenme yoktur; o makale yalnız Transformer'ı anlatır. Kendi kendine öğrenme daha önce de vardı. Transformer'ların bu süreçte kullanılması bütün süreci çok hızlandırdı. RNN'lerin pratikte en sevilmeyen özelliği kolay paralelleştirilememeleridir: kelimeleri teker teker vermek gerekir ve bu zaman alır. Transformer'lar kelimelerin hepsini tek seferde aldığı için çok daha hızlıdır. Hoca, öz-dikkat dışında teorik olarak büyük bir fark olmasa da Transformer'ların RNN'lere göre en önemli özelliğinin bu hız olduğunu söyledi.

Slayttaki görev listesi şöyledir (ekrandan, 01:44:40):

- Büyük miktarda eğitim verisi üretmek için üretilen yan görevler.
- İnsan etiketlemesine / emeğine ihtiyaç yok.
- Sonraki kelime(ler)i / cümleyi / film karesini tahmin.
- Gürültü ekleme
  - Maskelenmiş kelime(ler)i tahmin.
  - Sırası bozulmuş cümleyi düzeltme.
  - Harfleri bozulmuş kelimeyi düzeltme.
  - Resmin silinmiş kısmını tahmin.
  - vb.

Resim için düşünülürse: bir resmin bir tarafı kapatılır ve orası tahmin ettirilir. Elinizde bir resim varsa cevap zaten elinizdedir ve aynı resimden bunun gibi yüzlerce örnek üretilebilir. Hoca, kendi kendine öğrenme paradigmasının büyük modellerin önündeki veri kıtlığı problemini aşmasını sağladığını, Transformer'ların hızının ve internetteki verinin de buna denk geldiğini söyledi.

15.1 BART görevleri

Slayttaki BART şeması, özgün cümle "ABC.DE." (iki cümle) için şu bozulmuş girişleri gösterir; hepsinde çıkış özgün cümledir:

Görev	Bozulmuş giriş
Token Masking	A_C._E.
Token Deletion	A.C.E.
Text Infilling	A_.D_E.
Sentence Permutation	DE.ABC.
Document Rotation	C.DE.AB

Hoca bunların hepsini, orijinal veriyi belli manipülasyonlarla bozup modelden orijinal veriyi yeniden oluşturmasını istemek olarak özetledi: bazı kelimeler maskelenir, bazı token'lar silinir, cümlelerin sırası değiştirilir ya da metin döndürülür.

16. Alt Kelimeler ve Byte Pair Encoding

Son aradan sonra hoca temsil problemine geçti. Veri kümelerinin boyutu çok büyüyünce, girdilerin birimi ne olacak sorusu önem kazandı.

16.1 Kelime, karakter, alt kelime

- Kelime düzeyi: sözlük boyutu çok büyür. Ortalama bir derlemde 1 milyon farklı kelime (toplam değil, farklı kelime sayısı) olabilir; bu kadar token karşımıza çıkabilir. Bu kötüdür.
- Karakter düzeyi: "araba" dendiğinde a, r, a, b, a'nın her birinden bir anlam çıkarmak gerekir. Bu da optimal değildir.
- Alt kelime (subword): ikisinin arasında bir orta yoldur.

Slayttaki örnek (ekrandan, 02:10:00):

```
Kelime     : Geleceğim
Karakter   : G e l e c e ğ i m
Alt kelime : Gele #ceğ im
```

Alt kelimelerle verildiğinde, kelimeler parçalarına ayrıldığı için sözlük boyutu aşırı artmaz. "Gelmeyeceğim" gibi bir kelime de buna çok benzeyen alt kelimelerle kodlanabilir.

16.2 Sözlük dışı kelime problemi

Alt kelimeler sözlük dışı kelime (out of vocabulary, OOV) problemini de çözer. Bu problem, test kümesinde bulunan bir kelimenin eğitim kümesinde hiç olmamasıdır; o kelime nasıl temsil edilecektir? Özellikle Türkçe gibi morfolojik açıdan zengin, parçaların birleşerek kelimeleri oluşturduğu dillerde bu çok büyük bir problemdir: eğitimde "geleceğim" olabilir ama "gelmeyeceğim" olmayabilir. Kelimeler parçalarla temsil edilirse bu problemden kurtulunur, çünkü alt kelimelerin en alt seviyesinde harflerin kendisi de yer alır. En kötü durumda harfler arka arkaya getirilerek kelime yine oluşturulur; böylece her kelime temsil edilebilir.

16.3 Byte Pair Encoding

Alt kelimeleri belirlemek için kullanılan en temel yaklaşım bir veri sıkıştırma yönteminden gelir: Byte Pair Encoding (BPE). Hoca bunun birkaç farklı versiyonu olduğunu, ama özünün şu olduğunu anlattı: sıklıkla geçen bitişik ikililere (bigram) bakılır; en sık geçen ikili yeni bir sembolle değiştirilir; bu, tekrar eden ikili kalmayana kadar sürdürülür. Slayttaki örnek (ekrandan, 02:08:40):

```
aaabdaaabac
ZabdZabac    (Z=aa)
ZYdZYac      (Y=ab Z=aa)
XdXac        (X=ZY Y=ab Z=aa)
```

Adım adım:

- Verilen dizi: aaabdaaabac.

- 1. adım: bitişik ikililerin sıklıkları sayılır. “aa” örtüşmeli sayımla 4 kez geçer (her “aaa” içinde iki kez); “ab” 2 kez, diğerleri birer kez. En sık ikili “aa”dır ve Z ile değiştirilir. Her “aaa” grubunda yalnız bir “aa” değiştirilebilir, bir “a” kalır: ZabdZabac.
- 2. adım: ZabdZabac’ta “ab” 2 kez geçer ve Y ile değiştirilir: ZYdZYac.
- 3. adım: ZYdZYac’ta “ZY” 2 kez geçer ve X ile değiştirilir: XdXac.
- Artık birden fazla geçen ikili kalmaz ve işlem durur.

Sonuçta sözlük Z (= aa), Y (= ab) ve X (= ZY = aaab) sembollerinden ve bunların karşılıklarından oluşur.

> **Ders sonrası not:** 2. adımda “Za” ikilisi de “ab” gibi 2 kez geçer; ikisi eşit sıklıktadır ve slayt “ab”yi seçmiştir (python ile sayıldı). Ayrıca hoca bu örnekteki en sık ikilinin “aa” olmasının yapay örnekten kaynaklandığını, Türkçede en sık bigram’ın kesinlikle “aa” olmadığını söyledi.

Bu işlem gerçek bir derleme uygulandığında, frekansı yüksek alt kelimeler bulunur ve sözlüğe girer. Dil modellerinin tokenizer’larında bu parçalar şöyle görünür: başında işaret olmayan bir parça bir kelime başıdır; başında “##” gibi bir işaret olan parça bir kelimeden sonra gelen bir parçadır, yani bir ektir.

Bu sözlükler çok büyük derlemlerle hazırlanır. BERT’lerin 32K, 64K gibi sözlükleri vardır; hoca Türkçe için 32K ve 64K’lık sürümler olduğunu, 128K’yı bilmediğini ve gerek olmadığını düşündüğünü söyledi. Sözlük boyutu arttırıldığında içine daha fazla bütün kelime ve kelime parçası girer, ama 32K’dan 64K’ya çıkıldığında performans açısından çok büyük farklılıklar olmaz.

BPE, makine çevirisinde nadir kelimeleri temsil etmek üzerine önerilen yapıda (“Neural Machine Translation of Rare Words with Subword Units”) ilk popülerliğini kazandı. Hoca bunu makine çevirisinin lokomotif problem olmasının bir başka örneği olarak gösterdi. WordPiece ve SentencePiece bu yöntemin gerçeklemeleridir; token’ları ayırmak ve sonra birleştirmek için kullanılan kütüphanelerdir. Hoca OpenAI’nin de bir gerçeklemesi olduğunu, ama adını o an hatırlayamadığını söyledi.

Slaytta ayrıca şu soru vardır: Türkçe için alt kelimeler sadece OOV için değil, normal kelimeler için de çok önemlidir; neden? Hoca, alt kelimeler sayesinde farklı ifadelerin ortaklıklarının görülebildiğini söyledi (02:12:18).

> **Ek bilgi (derste anlatılmadı):** BERT’in kullandığı WordPiece, büyük olasılıkla BPE’den farklı olarak birleştirilecek ikiliyi yalnız sıklığa göre değil, birleştirmenin eğitim verisinin olabilirliğini (likelihood) ne kadar arttırdığına göre seçer. SentencePiece ise metni önceden kelimelere ayırmadan, boşlukları da bir sembol olarak ele alarak ham metin üzerinde çalışır.

17. Alanın Genel Resmi

Hoca son olarak bağlam tabanlı modellerin genel bir resmini gösterdi ve bunun biraz Google merkezli çizilmiş, 2020’den kalma bir resim olduğunu söyledi. 2023’e gelindiğinde resmin epey değiştiğini, alanın çok hızlı değiştiğini belirtti. Özellikle GPT tarafına bir yönelme olduğunu, sonraki token tahmininin standart yaklaşım haline geldiğini ve diğer ilginç ya da gösterişli kendi kendine öğrenme görevlerine göre sonraki token tahminiyle epey sağlam sonuçlar alınabildiğini söyledi. Slaytta Türkçe için kullanılabilecek modeller ve kaynaklar da vardır; hoca BERTurk gibi modellerin, önceki öğrencilerin çalışmalarının ve internetteki çeşitli kaynakların burada görülebileceğini söyledi. Değişimin giderek hızlandığını, önceden tahmin edilemeyen bir durum olduğunu ve bundan en çok kullanıcıların faydalanmasını umduğunu ekledi.

Transformer’ların yapısının daha ayrıntılı anlatımı için hoca internetteki kaynaklara ve kendi ders sayfasındaki Transformer kursuna yönlendirdi.

Örnek Sorular

Soru 1 (ekrandan, 00:11:30): Resimlerdeki insanları kadın/erkek, gözlüklü/gözlüksüz, uzun/kısa diye belirlemek istiyoruz. 8 sınıf tanımlamak yerine 3 ikili sınıflandırıcı kullanmanın avantajı nedir? N sınıf kaç alt özellikle ifade edilebilir?

Cevap (derste verildi): 8 sınıf tanımlanırsa her sınıftan örneğe ihtiyaç olur. 3 alt özelliği öğrenen 3 ikili sınıflandırıcı tanımlanırsa, eğitim kümesinde örneğin uzun, kadın, gözlüksüz bir örnek olmasa bile bu kombinasyon tanınabilir. d alt özellik 2^d kombinasyon ifade ettiği için, birbirini dışlamayan özellikler taşıyan N sınıf $\log_2 N$ alt özellikle ifade edilebilir.

Soru 2 (ekrandan, 00:15:10): 1024 sınıflı bir problemde $d \times d$ boyutlu resimler sınıflandırılacaktır. Ara katmansız klasik yaklaşımda ve 10 alt özellikli dağıtık yaklaşımda kaç parametre öğrenilir? $d = 128$ için karşılaştırınız.

Cevap (derste verildi, ders sonrası düzeltmeyle): Klasik: $(d \cdot d) \cdot 1024$. Dağıtık: $(d \cdot d) \cdot 10 + 10 \cdot 1024$. $d = 128$ için klasik $128^2 \cdot 1024 = 2^{24} = 16.777.216$ (derste "yaklaşık 4 milyon" denildi; doğrusu yaklaşık 16,8 milyon), dağıtık $163.840 + 10.240 = 174.080$ (yaklaşık 180 bin). Katman sayısı artmasına rağmen parametre sayısı yaklaşık 96 kat azalır.

Soru 3 (ekrandan, 00:40:00): $h_t = f(x_t, h_{t-1})$ olduğuna göre h_3'ü yalnız girişler ve h_0 cinsinden yazınız. h_3 hangi girişlere bağlıdır?

Cevap (derste verildi): $h_3 = f(x_3, h_2) = f(x_3, f(x_2, h_1)) = f(x_3, f(x_2, f(x_1, h_0)))$. h_3; x_3, x_2, x_1 ve h_0'a, yani o ana kadarki bütün girişlere bağlıdır.

Soru 4 (ekrandan, 00:37:40): "Ali bugün okula giderken yolda arkadaşı Hilmiyi gördü." cümlesinden, girişi 5 kelime ve çıkışı sonraki kelime olan sabit uzunluklu bir ağ için eğitim örnekleri üretiniz. Bu yapının RNN'den farkı nedir?

Cevap (derste verildi): "Ali bugün okula giderken yolda → arkadaşı", "bugün okula giderken yolda arkadaşı → Hilmiyi", "okula giderken yolda arkadaşı Hilmiyi → gördü". Sabit yapıda pencere uzunluğu önceden seçilmek zorundadır ve pencereden önceki kelimeler (örneğin son tahminde "Ali") çıkışı etkileyemez. RNN'de girişler teker teker verilir ve uzunluk sınırı teoride sonsuzdur.

Soru 5 (ekrandan, 00:50:00): RNN ile bir sonraki harf tahmininde one-hot kodlama kullanıldı. Bir sonraki harf yerine bir sonraki kelimeyi tahmin etmek için one-hot kullansak ne olur? Alternatifi nedir?

Cevap (derste verildi): Karakter düzeyinde bile yüze yakın sembol gerekir; kelimelere geçildiğinde one-hot vektörün boyutu (sözlük boyutu) çok artar. Karakter ile kelimenin iyi yanlarını birleştiren alternatif, alt kelimelerin (subword) kullanılmasıdır.

Soru 6 (derste sözlü, 01:13:38): Kod çözücüye her adımda C'yi ve y_{t-1}'i ayrıca vermek teorik olarak gerekli midir?

Cevap (derste verildi): Hayır. C zaten kodlayıcının son durumu, y_{t-1} ise zaten h'den üretilmiş bir değerdir; temel yapı evrensel olarak yeterlidir. Ama bu eklemelerin pratikte öğrenmeyi kolaylaştırdığı görülmüştür.

Soru 7 (ekrandan, 01:30:00): "Garajımda 300'le gidebilen bir jaguar var." ve "Ormanda beni jaguar kovaladı." cümlelerindeki "jaguar" kelimesi word2vec'te ve bağlam tabanlı bir modelde nasıl temsil edilir?

Cevap (derste verildi): word2vec'te "jaguar"ın bağlamdan bağımsız tek bir vektörü vardır ve bu vektör hem arabaya hem hayvana yakındır. Bağlam tabanlı modelde (öz-dikkat sayesinde) vektör cümledeki diğer kelimelere (garaj, 300'le gidebilmek; orman, kovalamak) göre hesaplanır ve iki cümlede farklı olur. Bu yüzden vektör saklanamaz; model saklanır.

Soru 8 (ekrandan, 02:08:40): "aaabdaaabac" dizisine Byte Pair Encoding uygulayınız.

Cevap (derste verildi): aaabdaaabac → ZabdZabac (Z=aa) → ZYdZYac (Y=ab) → XdXac (X=ZY). Tekrar eden ikili kalmayınca durulur. Sözlük: Z = aa, Y = ab, X = aaab.

Soru 9 (ekrandan, 02:10:00): Türkçe için alt kelimeler sadece OOV için değil, normal kelimeler için de çok önemlidir. Neden?

Cevap (derste kısmen verildi): Hoca, alt kelimeler sayesinde farklı ifadelerin ortaklıklarının görülebildiğini söyledi.

Çözüm (derste verilmedi): Türkçe eklemeli bir dil olduğundan aynı kökten çok sayıda kelime türer. Bu kelimeler ortak alt kelimeleri paylaşınca (örneğin "Geleceğim" ve "Gelmeyeceğim" ortak parçalarla kodlanınca) model, bütün kelimeleri ayrı semboller olarak görmek yerine bu ortaklığı kullanabilir ve her biçim için ayrı ayrı çok örnek görmesi gerekmez.

Dersin Toparlaması

Bu derste Bölüm 5'e, bağlam farkında sinirsel dil modellerine başlandı. Önce bu modellerin yapı taşları anlatıldı. Dağıtık temsil, sınıfları alt özelliklerin kombinasyonu olarak görerek parametre sayısını azaltır ve görülmemiş kombinasyonların tanınmasını sağlar; CNN'lerin alt katmanlarında öğrenilen ortak ilkel özellikler de öğrenme transferinin temelidir. Kapsül ağları bu parçaların göreceli konumlarını da işe katar. Ardından sıralı girişler için RNN'ler, bunların bütün geçmişi nasıl taşıdığı, uygulama türleri ve W_{xh}, W_{hh}, W_{hy} ağırlıkları görüldü. LSTM ve GRU, neyin hatırlanıp neyin unutulacağını öğrenen RNN türevleridir. Makine çevirisi için kodlayıcı-kod çözücü mimarisi, bunun iyileştirmeleri (ters giriş, C ve y_{t-1}'i kod çözücüye vermek) ve her çıkış için girişlere öğrenilmiş ağırlıklar veren dikkat mekanizması anlatıldı.

Transformer'lar RNN'i kaldırarak paralelleşmeyi ve öz-dikkati getirdi; bir kelimenin temsili artık içinde geçtiği cümleye göre hesaplanır. Bu, word2vec gibi bağlamdan bağımsız temsillerden bağlam tabanlı dil modellerine (TagLM, ELMo, GPT, BERT, RoBERTa, ALBERT, ELECTRA, BART) geçişi sağladı. Bu modellerin büyük veriyle eğitilebilmesini, insan etiketi gerektirmeyen kendi kendine öğrenme görevleri mümkün kıldı. Son olarak sözlük boyutu ve OOV problemine çözüm olan alt kelimeler ve Byte Pair Encoding anlatıldı. Hoca bu modellerin güncel hallerinin dönem sonuna doğru daha ayrıntılı ele alınacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.